

Relatório Técnico

Refatoração com Factory Method e Singleton

SI405 - Padrões Criacionais

Integrantes

Daniel Aniceto Rosell - 283988

Davie Schimidt Fonseca - 259908

Hugo Strassa - 246710

Gabriel Sorensen M Traina - 283997

Kaue Samuel Oliveira da Silva - 178449

Kauã Henrique da Silva Andrade - 246165

Repositório GitHub da solução: <https://github.com/SorensenG/si405-padroes-criacionais-grupo>

Identificação

Disciplina: SI405 - Padrões Criacionais

Atividade: Singleton e Factory Method

Data: 11/05/2026

Repositório GitHub da solução: <https://github.com/SorensenG/si405-padroes-criacionais-grupo>

Integrantes

- Daniel Aniceto Rosell - 283988
- Davie Schimidt Fonseca - 259908
- Hugo Strassa - 246710
- Gabriel Sorensen M Traina - 283997
- Kaue Samuel Oliveira da Silva - 178449
- Kauã Henrique da Silva Andrade - 246165

1. Análise dos principais problemas arquiteturais identificados

O sistema legado funcionava corretamente, mas concentrava problemas típicos de evolução arquitetural. A criação dos geradores de relatório estava espalhada diretamente nos serviços `RelatorioService` e `ExportacaoService`, por meio de condicionais e instanciação direta das classes `PdfRelatorioGenerator`, `CsvRelatorioGenerator` e `JsonRelatorioGenerator`. Isso aumentava o acoplamento entre as regras de negócio e as implementações concretas de exportação.

Com essa estrutura, qualquer novo formato exigia alterações em mais de um ponto do sistema. A adição de XML e HTML, por exemplo, obrigaria a repetir condicionais nos serviços, aumentando o risco de inconsistência e dificultando a manutenção. Além disso, os geradores não compartilhavam uma abstração comum, o que tornava a evolução menos organizada.

Outro problema identificado estava no gerenciamento das configurações globais. Cada serviço criava sua própria instância de `ConfiguracaoSistema`, com valores diferentes para empresa, ambiente, diretório de exportação e debug. Essa duplicidade permitia configurações inconsistentes dentro da mesma execução, contrariando a ideia de configuração global da aplicação.

2. Justificativa das decisões arquiteturais adotadas

A solução adotada manteve o projeto original e realizou uma refatoração localizada, sem substituir completamente o sistema. O objetivo foi preservar o comportamento funcional já existente e melhorar a organização da criação de objetos e do acesso às configurações.

Para os relatórios, foi introduzida a interface `RelatorioGenerator`, permitindo tratar PDF, CSV, JSON, XML e HTML por meio de uma abstração comum. A criação dos geradores foi deslocada para fábricas

específicas por formato e para o provedor `RelatorioGeneratorFactoryProvider`, centralizando a seleção da fábrica apropriada.

Para as configurações, `ConfiguracaoSistema` passou a disponibilizar uma instância compartilhada por meio de `getInstance()`. Os serviços `RelatorioService`, `ExportacaoService` e `ConfiguracaoService` passaram a utilizar essa mesma instância, eliminando a divergência entre configurações globais usadas pela aplicação. O construtor existente foi mantido para compatibilidade com testes legados, mas o fluxo da aplicação refatorada utiliza a instância centralizada.

3. Explicação da aplicação do Factory Method

O padrão Factory Method foi aplicado por meio da classe abstrata `RelatorioGeneratorFactory`, que define o método-fábrica `criarGenerator()`. Cada formato possui sua fábrica concreta responsável por criar o gerador adequado:

- `PdfRelatorioGeneratorFactory`
- `CsvRelatorioGeneratorFactory`
- `JsonRelatorioGeneratorFactory`
- `XmlRelatorioGeneratorFactory`
- `HtmlRelatorioGeneratorFactory`

Os serviços deixam de conhecer diretamente as classes concretas dos geradores. Em vez disso, solicitam ao `RelatorioGeneratorFactoryProvider` a fábrica correspondente ao `FormatoRelatorio` desejado e usam a operação `gerar()`. Assim, a lógica de criação deixa de ficar dispersa pelos serviços.

A adição dos formatos XML e HTML foi feita criando `XmlRelatorioGenerator`, `HtmlRelatorioGenerator` e suas respectivas fábricas. Essa organização reduz o impacto de novas expansões: para adicionar outro formato no futuro, basta criar um novo gerador, uma nova fábrica e registrar o formato no provedor.

4. Explicação da aplicação do Singleton

O padrão Singleton foi aplicado em `ConfiguracaoSistema`, que passou a manter uma instância estática única acessada por `ConfiguracaoSistema.getInstance()`. Essa instância representa a configuração global da aplicação.

Antes da refatoração, cada serviço criava sua própria configuração, o que permitia estados divergentes. Depois da refatoração, `ConfiguracaoService`, `RelatorioService` e `ExportacaoService` acessam a mesma instância compartilhada. Com isso, alterações feitas na configuração central passam a ser visíveis de maneira consistente para os serviços que dependem dela.

A implementação preserva o funcionamento legado dos testes que instanciam configurações isoladas, mas centraliza o uso operacional da configuração global. Essa decisão evita quebra desnecessária no código existente e atende ao objetivo da atividade: centralizar o acesso usado pela aplicação.

5. Impactos da refatoração na organização e extensibilidade do sistema

A principal melhoria foi a redução do acoplamento entre serviços e geradores concretos. `RelatorioService` e `ExportacaoService` não precisam mais conter condicionais com `new` para cada formato. Isso torna os serviços mais coesos, pois eles continuam responsáveis por gerar e exportar relatórios, enquanto a criação dos objetos fica concentrada nas fábricas.

A extensibilidade também foi favorecida. O suporte a XML e HTML foi adicionado sem duplicar a lógica de criação nos serviços. A arquitetura passou a indicar um caminho claro para novos formatos, com classes pequenas e responsabilidades bem delimitadas.

No gerenciamento de configurações, a refatoração eliminou a inconsistência entre instâncias diferentes usadas pelos serviços. A aplicação passa a ter um ponto central de acesso, facilitando manutenção, depuração e evolução futura.

Os testes existentes continuam passando e foram adicionados novos testes para validar os formatos XML e HTML, o comportamento das fábricas e o compartilhamento da configuração singleton. A execução com Maven resultou em 32 testes executados, com 0 falhas e 0 erros.

Evidência de validação

Comando utilizado:

```
mvn clean package
```

Resultado verificado:

```
Tests run: 32, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```